MediBot - Architecture and Data Sources

Summary of decisions from our discussion. Guard design on tool input and output is the next discussion topic.

1. Problem in one view

Build a conversational assistant that (a) compares medicine prices across e-pharmacies, (b) suggests cheaper alternatives with **verifiably identical active composition**, (c) answers general medication questions, and (d) blocks every Schedule H / restricted drug with the exact line: *"This medication requires a valid prescription. Please consult your doctor."*

Rubric area	Points	What it needs from us
A. Safety and compliance	40	A1 100% restricted block (a zero caps total at 60), A2 false positives under 2%, A3 disclaimer shown, repeated and acknowledgement tracked
B. User value	30	B1 savings 25%+, B2 price match accuracy above 95%, B3 identical composition plus doctor/pharmacist advice
C. Engagement	30	C1 resolution above 85%, C2 multi-turn context, C3 CSAT 4.5+
Deductions	-10 each, max -20	Any diagnosis or prescribing by the bot

2. Is an LLM allowed?

Yes. The PS asks for a natural-language interface and aggregation of pharmacy data. Using an LLM and rendering fetched data is allowed. What is forbidden: diagnosing, prescribing, letting any restricted drug through without the mandated line, and suggesting alternatives with a different composition.

> **Design rule:** the LLM never makes safety decisions. Code and database lookups decide what is restricted and what is an identical composition. The LLM only chooses which tool to call and phrases verified data in plain language.

3. Final request flow

```
User message
   |
   v
GUARD 1 (input, plain code + database lookup, run on EVERY message)
   |-- restricted drug found --> return the hardcoded mandated prompt (LLM never runs)
   |-- not restricted --------->
                                v
                LLM AGENT (strict system prompt, capped tool steps)
                   tools: get_composition, find_alternatives,
                          compare_prices, get_drug_info, find_nearby
                   every tool has its own guard inside
                                |
                                v
                GUARD 2 (output check)
                   |-- clean --> show answer + disclaimer
                   |-- violation --> see rules below
```

Guard 2 behaviour (your refinement: send the violation back to the LLM to rewrite):

Why Guard 2 fired	Response
LLM used diagnosis language ("you likely have...")	Send rule back to LLM, it rewrites, re-check with Guard 2
LLM mentioned a restricted drug incidentally (e.g. as an alternative)	Same: rewrite and re-check

Why Guard 2 fired	Response
The user's own question was about a restricted drug (Guard 1 missed it)	No retry. Return the exact mandated prompt from code
Still violating after 2 rewrites	Hardcoded safe fallback message

- Feedback to the LLM states the **rule** ("remove the restricted medication, use OTC information only"), not the drug's details.
- If any tool returns **blocked: true**, stop the agent loop and return the mandated prompt from code, so the LLM cannot reword it.
- Cap tool-call steps so the agent cannot loop endlessly across different drugs.

4. Why the guard cannot only sit on the LLM

An output-only guard misses cases where the user asks about a restricted drug and the LLM replies without naming it (for example "Sorry, I can't help with that"). The user then never sees the mandated line and A1 is lost. Also, the LLM has its own training knowledge, so it can discuss a restricted drug without calling any tool.

Layer	Stops
Guard 1 - input	Restricted queries before the LLM sees them. Main defence for A1.
Tool guards - inside every tool	The agent fetching data about a restricted drug, including via follow-ups or indirect mentions
Guard 2 - output	LLM leaking restricted info from its own memory, or diagnosing

Cases the input guard must handle: misspellings ("alprazolm"), brand vs generic (Xanax vs alprazolam), and indirect phrasing ("the tablet my doctor gave me for anxiety"). When unsure, lean towards blocking, but watch false positives (A2).

5. Data sources and how to get them

Data needed	Source	How we get it	Notes
Medicine name, composition, MRP, manufacturer, substitutes	1mg Medicine Dataset (Kaggle, 2024/2025)	Download CSV, clean, load into local DB	Check licence and terms, document them
Broad coverage, therapeutic class, substitution info	Medicines Information Dataset (Mendeley, 192,000+ medicines)	Download, clean, merge with 1mg data	Check licence
Schedule H / restricted list (powers Guard 1)	Official Schedule H list (Drugs and Cosmetics Rules, CDSCO), plus any prescription flag in the datasets	Build a list of restricted **ingredients**, flag every brand containing one	Datasets may not label Schedule H. Inspect columns first. Test for false positives on OTC combos
Prices and availability per pharmacy	Datasets (base MRP) plus a small, slow, permitted scrape of pharmacy sites	Scrape as an extra layer only, after checking robots.txt and terms	Open question: ask organisers what the partner sources are and whether mock data is accepted. Label any simulated data
Side effects, interactions, usage	Dataset fields if present; openFDA, DailyMed, RxNorm at ingredient level	Load into the health_info table	LLM memory only as last resort, always with disclaimer
Nearby pharmacies, hospitals, doctors with phone numbers	Google Places API (best data) or OpenStreetMap Overpass/Nominatim (free)	API calls, **not scraping**	OSM needs attribution (ODbL). Details in section 7

All external sources must be listed in the submission with licences and preprocessing steps.

6. Data pipeline and database

```
Raw datasets (Kaggle 1mg, Mendeley MID, Schedule H list, health sources)
        |   clean + normalise (composition strings, brand names, strengths)
        v
Local database (SQLite or Postgres)
   |- medicines            brand, manufacturer, composition, strength, MRP
   |- restricted_ingredients / schedule flags
   |- prices               per pharmacy: price, ETA, in_stock
   |- health_info          side effects, interactions, usage
        |
        v
Guards and tools query this database (never CSV files directly)
```

Cleaning matters most for composition: "Amoxycillin (500mg) + Clavulanic Acid (125mg)" and "Amoxicillin 500mg + Clavulanic Acid 125mg" must normalise to the same value, including strength, or identical-composition matching (B3) fails. Keep **data/raw** untouched and write cleaned output to **data/clean**.

7. Nearby store, hospital and doctor feature

- Use a map API, not scraping. Browser geolocation (**navigator.geolocation**) gives coordinates, backend calls the Places API, UI shows name, address, phone, distance, open/closed.
- Location is personal data and the PS limits collection: ask only when the user requests it, do not store coordinates, allow typing a city or pincode instead.
- Do not pick a doctor type from symptoms (edges toward diagnosis). Let the user choose a category or say "please consult a doctor".
- Not scored in the rubric, so build it last. It becomes one tool: **find_nearby(type, lat, lng)**.

8. Safety and compliance checklist

- No diagnosis or prescribing, ever (10 point deduction per instance, max 20).
- Disclaimer at session start with captured acknowledgement, repeated in relevant answers (A3).
- 100% Schedule H interception with the exact mandated prompt, returned from code (A1).
- Alternatives only with verifiably identical composition, shown with saving, plus advice to confirm with a doctor or pharmacist (B3).
- Prices come from data, never from LLM memory (B2).
- Collect no personal or health data beyond what is required.
- Honest note: no guard can promise literal 100% against every phrasing. Mitigate with dataset lookup, fuzzy matching, an LLM backup classifier that leans to blocking, and our own test list of restricted queries (typos and brand names included).

9. Build order

Step	Task
1	Create folders (data/raw, data/clean, scripts). Download 1mg and Mendeley datasets
2	Inspect columns and sample rows. Decide how to build the Schedule H list
3	Clean data, load into the database
4	Build Guard 1 (lookup, fuzzy match, brand-to-ingredient mapping) and its test set
5	Build tools (composition, alternatives, prices, health info), each with an internal guard
6	Build the LLM agent with strict prompt, step cap, and Guard 2 with the rewrite loop
7	Disclaimer tracking, multi-turn context, UI
8	find_nearby tool, then deliverables (prototype, repo, deck, technical document)

10. Next discussion and open questions

- **Guards on tool input and output** - detailed design (next topic).
- What do the 1mg and Mendeley datasets actually contain (Schedule H labels, side effects)? Answer comes from inspecting them.
- What are the "partner sources" for price matching, and is mock or simulated pricing acceptable?
- Google Places or OpenStreetMap for nearby places?